



UNIX SHELL

Group 09

OBJECTIVE

- Implement a UNIX-style interactive shell
- Support file and directory operations
- Implement file descriptor abstraction
- Enable I/O redirection (<, >, >>)
- Implement pipelines (|)
- Support recursive directory operations
- Maintain process isolation

SYSTEM ARCHITECTURE OVERVIEW

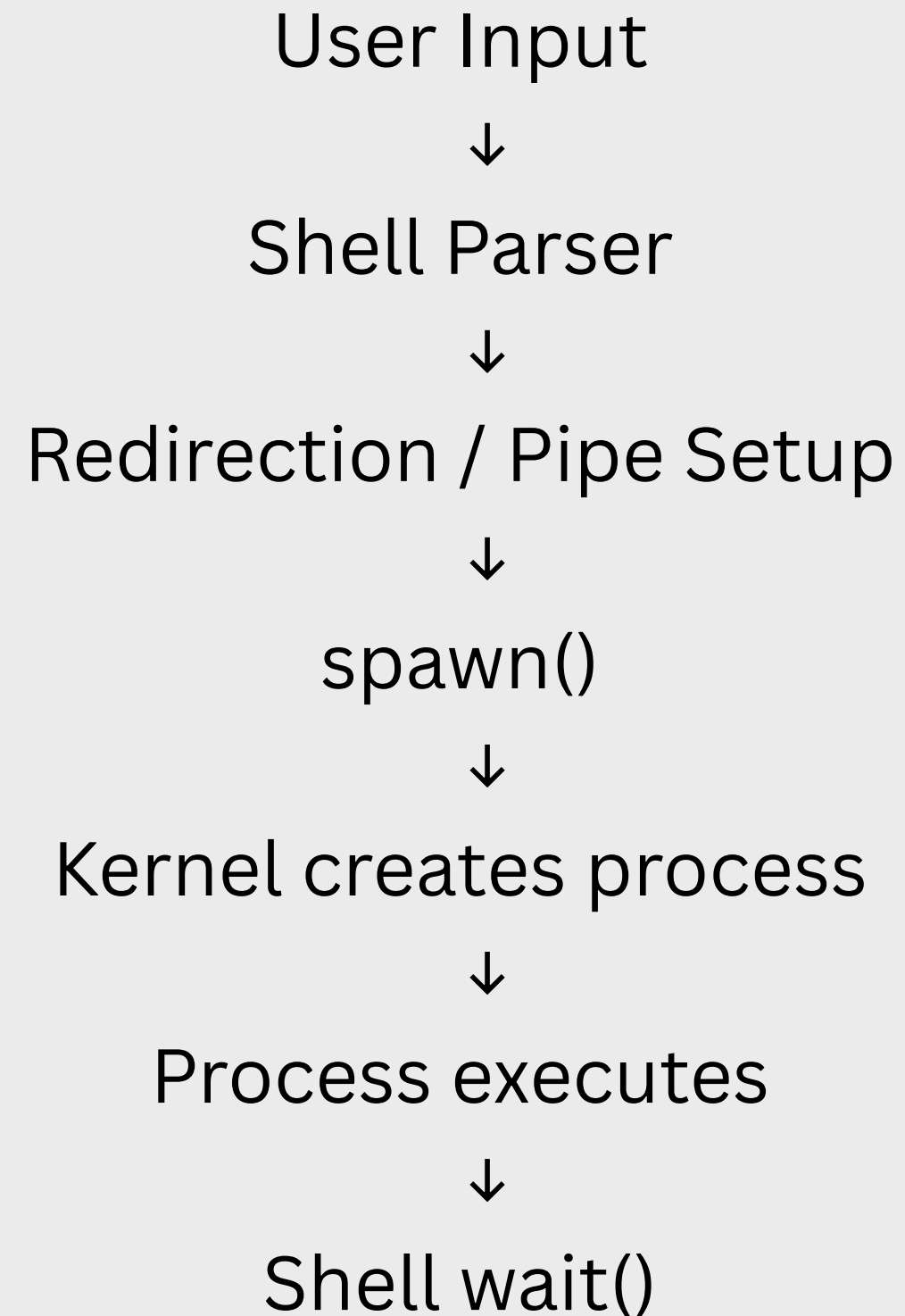
User Space:

- Shell process
- cp, mv, rm implementations
- Command parser

Kernel Space:

- File system (inode + directory)
- File descriptor table
- Pipe subsystem
- IPC system
- System call handler

SYSTEM ARCHITECTURE OVERVIEW



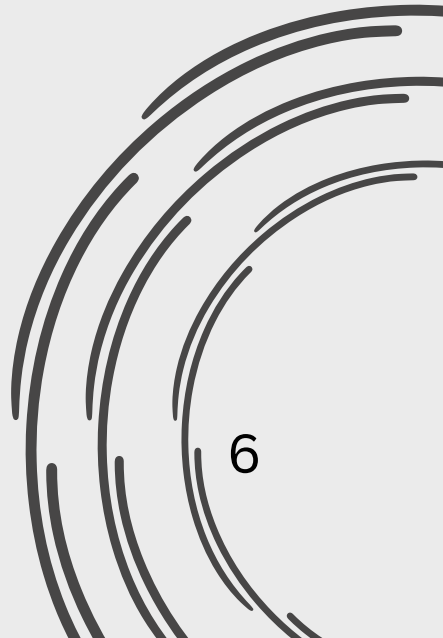


Kernel-Level Implementations

- File Descriptor Abstraction (per-process FD table with stdin/stdout/stderr)
- Unified I/O using SYS_read and SYS_write
- Extended SYS_spawn with custom input/output redirection
- Pipe-based Inter-Process Communication (SYS_pipe)
- Interactive UNIX-like Shell (REPL execution loop)
- Full File & Directory Operations (mkdir, cd, pwd, ls, touch, cat, cp, mv, rm)
- Recursive Operations Support (cp -r, rm -r)
- Input/Output Redirection and Multi-Stage Pipelines (<, >, >>, |)



IMPLEMENTATION





cat

- 'cat' is used to display file contents
 - Reads data from file and prints output to console
 - Can combine multiple files into single output
 - Demonstrates file reading using system calls
- 

cat Implementation

```
void sys_cat(tf_t *tf)
{
    int fd;
    int user_buf_addr;
    int buf_len;
    struct file *fp;
    struct inode *ip;
    int cur_pid = get_curid();
    int read_len;

    // 1. Get arguments from the syscall trap frame
    // arg2: file descriptor, arg3: user buffer address, arg4: buffer length
    fd = syscall_get_arg2(tf);
    user_buf_addr = syscall_get_arg3(tf);
    buf_len = syscall_get_arg4(tf);

    // 2. Retrieve the file pointer from the process's open file table
    fp = tcb_get_openfiles(cur_pid)[fd];

    // 3. Validation: Check if file exists and has a valid inode
    if (fp == 0 || fp->ip == 0) {
        KERN_DEBUG("sys_cat: Invalid file descriptor %d\n", fd);
        syscall_set_errno(tf, E_BADF);
        syscall_set_retval1(tf, -1);
        return;
    }

    ip = fp->ip;
```

```
    // 4. Validation: Ensure we are not trying to 'cat' a directory
    if (ip->type != T_FILE) {
        KERN_DEBUG("sys_cat: Cannot cat a directory\n");
        syscall_set_errno(tf, E_BADF);
        syscall_set_retval1(tf, -1);
        return;
    }

    // 5. Determine how much to read (limit by file size or buffer size)
    // Using the kernel's sys_buf to temporarily hold the data
    read_len = ip->size < PAGE_SIZE ? ip->size : PAGE_SIZE;
    if (read_len > buf_len) read_len = buf_len;

    // 6. Read data from the inode into the kernel buffer
    // Offset is usually tracked in the file pointer (fp->off)
    if (inode_read(ip, sys_buf[cur_pid], fp->off, read_len) != read_len) {
        syscall_set_errno(tf, E_MEM);
        syscall_set_retval1(tf, -1);
        return;
    }

    // 7. Copy the data from kernel space to user space addressing
    if (pt_copyout(sys_buf[cur_pid], cur_pid, user_buf_addr, read_len) != read_len) {
        syscall_set_errno(tf, E_MEM);
        syscall_set_retval1(tf, -1);
        return;
    }

    // 8. Update file offset and return success
    fp->off += read_len;
    syscall_set_errno(tf, E_SUCC);
    syscall_set_retval1(tf, read_len);
}
```




ls

- Used to list directory contents
 - Displays files and subdirectories in the current or specifies directory
 - Helps users view and navigate file system structure
 - Demonstrated directory traversal using system calls
- 

ls Implementation

```
void sys_ls(tf_t *tf)
{
    uint32_t off, inum;
    struct dirent de;
    uint32_t de_size;
    struct inode * p_inode;
    int user_buf_addr = syscall_get_arg2(tf);
    int buf_len = syscall_get_arg3(tf);
    int cur_pid = get_curid();
    int len;
    char * buf_p = sys_buf[cur_pid];
    struct inode* dp = (struct inode*)tcb_get_cwd(cur_pid);
    de_size = sizeof(de);
    for(off = 0; off < dp->size; off += de_size){
        if(inode_read(dp, (char *)&de, off, de_size) != de_size){
            // size is not legal
            KERN_PANIC("wrong in dir_lookup");
        }
        if(de.inum == 0){
            // free entry
            continue;
        }
        // TODO index may out of bound
        strncpy(buf_p, de.name, strlen(de.name, PAGESIZE));
        buf_p += strlen(de.name, PAGESIZE);
        *(buf_p++) = ' ';
        //dprintf("%s ", de.name);
    }
    *(buf_p - 1) = '\0';
    //dprintf("\n");
    len = buf_p - sys_buf[cur_pid] < buf_len? buf_p - sys_buf[cur_pid]: buf_len;
    pt_copyout(sys_buf[cur_pid], cur_pid, user_buf_addr, len);
    syscall_set_errno(tf, E_SUCC);
    syscall_set_retval1(tf, len);
}
```



THANK YOU